Visualização Geométrica, Rasterização e Recorte de Árvores Arteriais CCO

BCC327 – Computação Gráfica | Trabalho Prático 1

Brenda Gabrielle Alves Nascimento
Matrícula 24.1.4011

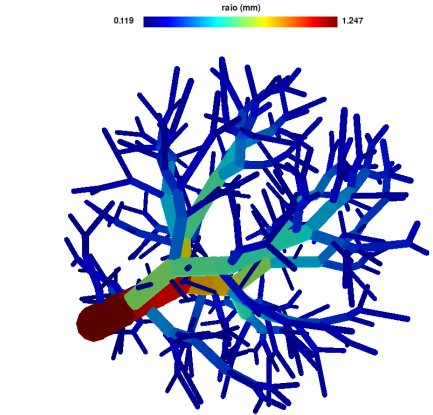
Departamento de Computação (DECOM)
Universidade Federal de Ouro Preto (UFOP)

2026.2 | Prof. Dr. Rafael Alves Bonfim de Queiroz

Roteiro da Apresentação

1. Contexto e Objetivos
2. Visualizador
3. Modo Didático
4. Discussão e Conclusão

Contexto: árvores arteriais CCO



tree3D_Nterm256 | perspectiva | vista isométrica | espessura + cor | exagero 2.0x | MSAA desligado

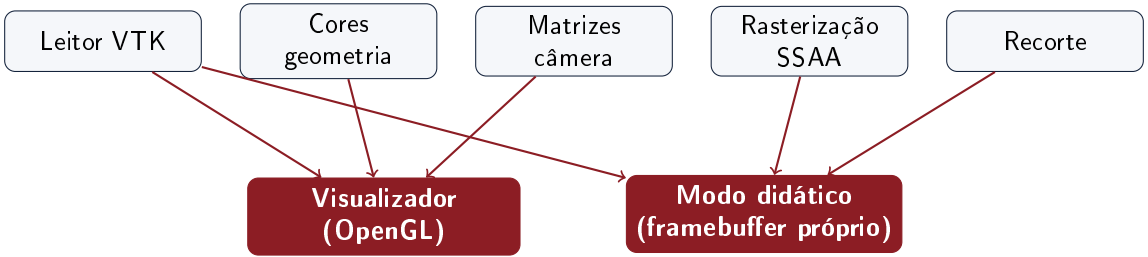
Arquivo VTK (POLYDATA)

`POINTS` coordenadas (x, y, z)
`LINES` segmentos `2 i j`
`CELL_DATA` `raio` por segmento (mm)

	Modelos	Pontos	Segmentos
2D	6	16 a 512	15 a 511
3D	6	32 a 1024	31 a 1023

- sempre segmentos = pontos − 1: uma árvore

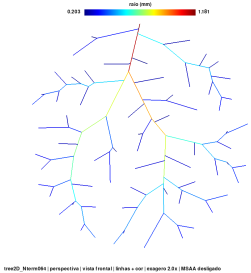
Arquitetura: dois programas, mesmos módulos



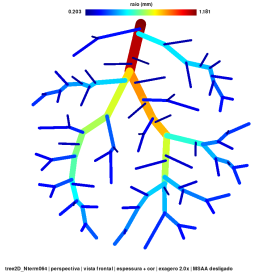
Ideia central

No modo didático, nenhum pixel vem da biblioteca: o pygame-ce só amplia a matriz NumPy escrita pelos nossos algoritmos.

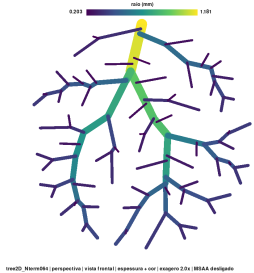
Primitivas e atributos visuais



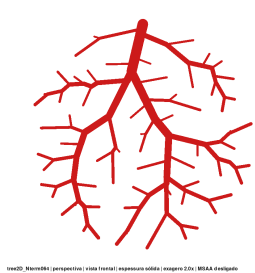
linhas



faixas + jet



faixas + viridis

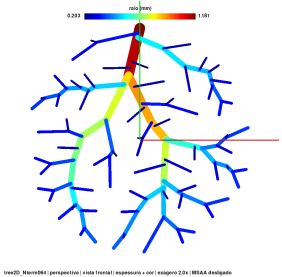


espessura sólida

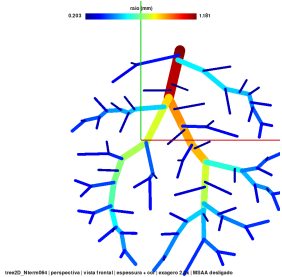
- cor: $t = \dfrac{r - r_{\min}}{r_{\max} - r_{\min}}$, interpolação RGB
- jet: contraste | viridis: uniforme

- faixa = 2 triângulos + discos nas pontas
- 3D: cilindros + esferas nas junções

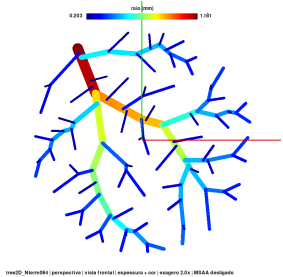
Transformações geométricas



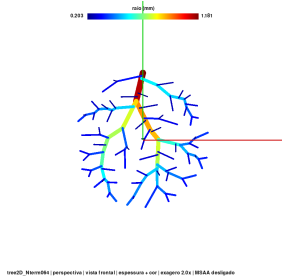
original



translação



rotação 35°



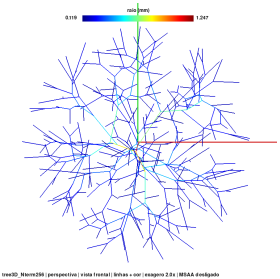
escala 0,6

Matrizes próprias em coordenadas homogêneas

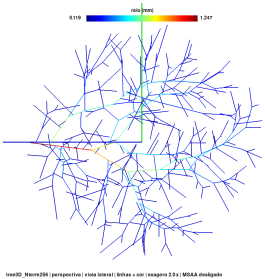
$$\mathrm{p}' = P \cdot V \cdot M \cdot \mathrm{p} \qquad M = T \cdot R_x R_y R_z \cdot S \cdot N$$

validadas contra `glOrtho`, `gluPerspective` e `gluLookAt`

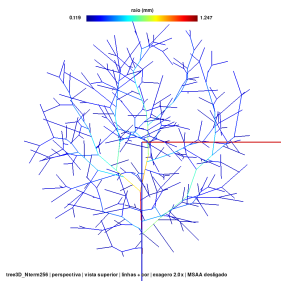
Observador: quatro vistas



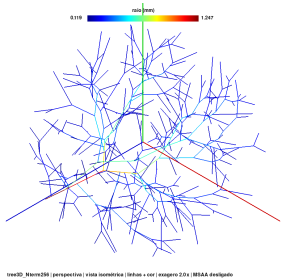
frontal



lateral



superior

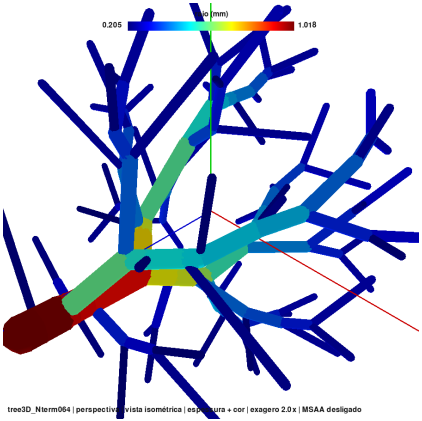


isométrica

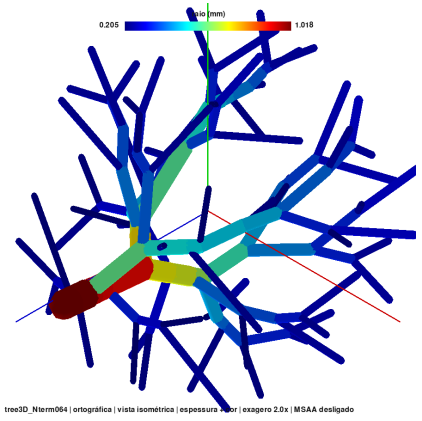
Espaço do mundo: referencial fixo da cena
Espaço da câmera: câmera na origem, olhando para $-z$

$$w = \frac{e - a}{\|e - a\|}, \qquad u = c \times w, \qquad v = w \times u$$

Projeção perspectiva × ortográfica



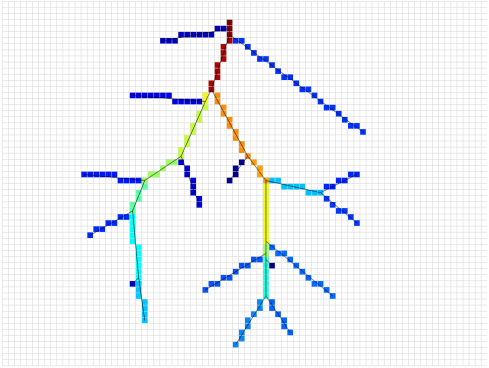
perspectiva: o tronco próximo fica maior



ortográfica: espessura não depende da distância

- diferença está na última linha: $(0, 0, -1, 0)$ copia a profundidade para w
- caixa ortográfica com $h = d \tan(\text{fovy}/2)$

Rasterização de segmentos: Bresenham



$p_0 = 2\Delta y - \Delta x$

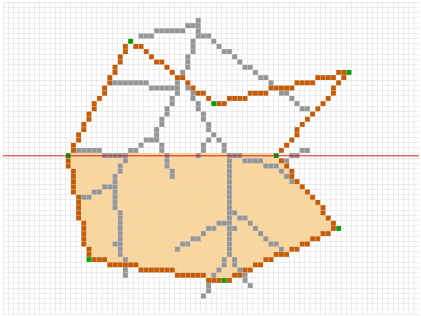
$p < 0$: mantém y, $p \mathrel{+}= 2\Delta y$

$p \geq 0$: $y{+}{+}$, $p \mathrel{+}= 2\Delta y - 2\Delta x$

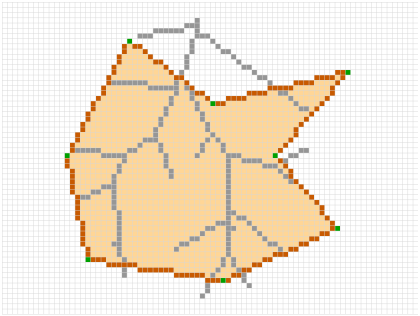
k	0	1	2	3	4	5	6	7	8	9	10
p_k	6	2	-2	14	10	6	2	-2	14	10	6
y	10	11	12	12	13	14	15	16	16	17	18

- $(20, 10)$ a $(30, 18)$: só inteiros
- 20 000 segmentos testados: erro $\leq 0{,}5$ pixel

Preenchimento: Scanline Fill



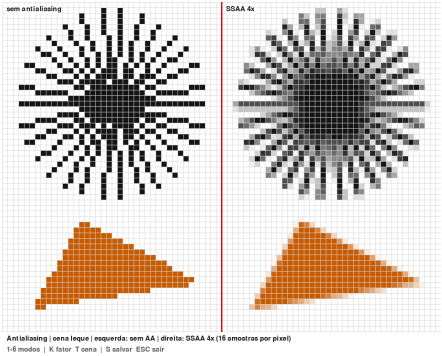
varredura em andamento



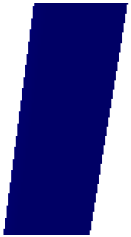
região de interesse preenchida

- tabela de arestas: $(y_{\max},\ x,\ 1/m)$
- arestas ativas ordenadas, preenche aos pares
- coerência entre linhas: $x \mathrel{+}= 1/m$

Aliasing e antialiasing



sem AA × SSAA 4× (16 amostras)



sem MSAA

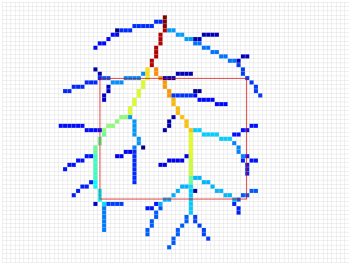


MSAA 4×

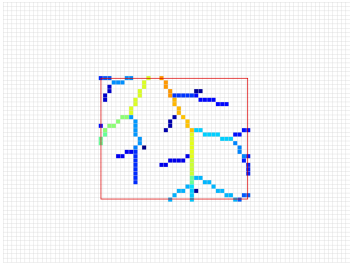
SSAA (nosso): grade $k\times$ maior + média dos blocos $k \times k$

MSAA (OpenGL): amostras de cobertura, cor calculada uma vez; FBO + `glBlitFramebuffer`

Recorte de segmentos



antes

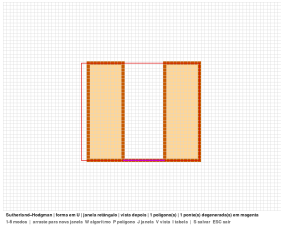


depois

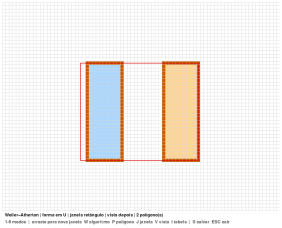
	Cohen–Sutherland	**Liang–Barsky**
Ideia	códigos de região de 4 bits	paramétrico, t_0 e t_1
Exemplo $(2,6)$–$(36,30)$	3 interseções (1 desperdiçada)	4 razões q/p, 2 pontos
100 000 segmentos	resultados idênticos	

- Cyrus–Beck: generaliza o Liang–Barsky para qualquer janela convexa

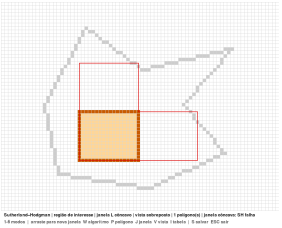
Recorte de polígonos: Sutherland–Hodgman × Weiler–Atherton



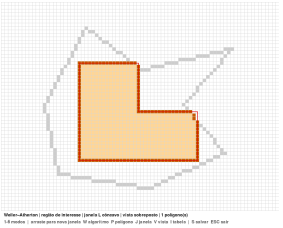
S–H: 1 polígono + ponte



W–A: 2 polígonos



S–H, janela côncava



W–A, janela côncava

Weiler–Atherton (diferencial)

Percorre as duas fronteiras trocando nas interseções: aceita polígonos côncavos e devolve regiões separadas.

O que foi implementado pelo grupo

Grupo	Biblioteca
Leitor VTK, mapas de cores	Janela, teclado, mouse, texto (pygame-ce)
Faixas 2D e tubos 3D com normais	Rasterização final dos triângulos (OpenGL)
Matrizes de transformação, câmera e projeção	Teste de profundidade, iluminação dos tubos
Bresenham, DDA, Scanline Fill	MSAA via FBO, VBOs
Supersampling (SSAA)	Operações vetoriais (NumPy)
Cohen–Sutherland, Liang–Barsky	
Sutherland–Hodgman, Weiler–Atherton	

Discussão

Decisões

- espessura como geometria, não `glLineWidth`
- recorte em coordenadas reais, depois rasterização
- MSAA por FBO (WSL sem janela multiamostrada)

Limitações

- Nterm512 em tubos lento com renderização por software
- iluminação da API (tema do TP2)
- W–A trata degenerados por deslocamento numérico

Conclusões

- 12 modelos lidos e representados por linhas, faixas e tubos
- transformações, vistas e projeções com matrizes próprias
- rasterização, preenchimento, antialiasing e recorte implementados e validados
- Weiler–Atherton como diferencial

Mensagem final

Separar visualizador e modo didático permitiu mostrar cada algoritmo pixel a pixel, sem caixa-preta.

Demonstração

Visualizador

- `E` representação, `C` cores
- mouse: rotação e zoom
- 1–4 vistas, `P` projeção
- `M` MSAA

Modo didático

- `1`, `2` Bresenham
- `3` + Espaço Scanline
- `4` SSAA
- `5` recorte de segmentos
- `6` + `P`, `W`, `J` recorte de polígonos

Referências

- SCHREINER, W.; BUXBAUM, P. F. Computer-optimization of vascular trees. *IEEE Transactions on Biomedical Engineering*, 1993.
- FOLEY, J. D. et al. *Computer Graphics: Principles and Practice*. Addison-Wesley, 1990.
- HEARN, D.; BAKER, M. P. *Computer Graphics, C Version*. Prentice Hall, 1997.
- BRESENHAM, J. E. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 1965.
- LIANG, Y.-D.; BARSKY, B. A. A new concept and method for line clipping. *ACM TOG*, 1984.
- SUTHERLAND, I. E.; HODGMAN, G. W. Reentrant polygon clipping. *CACM*, 1974.
- WEILER, K.; ATHERTON, P. Hidden surface removal using polygon area sorting. *SIGGRAPH*, 1977.

Comandos de Execução

```
pip install -r requirements.txt
python src/visualizador.py data/tree3D_Nterm256.vtk
python src/modo_didatico.py data/tree2D_Nterm032.vtk
python src/gerar_figuras.py        # regera as figuras
```

Observação

O README traz instalação, comandos de teclado e mouse e o roteiro de teste.

Obrigada!

Perguntas?

Brenda Gabrielle Alves Nascimento | 24.1.4011